# Shiv Nadar University Chennai

**CS3807 – Deep Learning Laboratory**

**Experiment 2**

**Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification**

Ankith U Davey 24011101004

| | | |
|---|---|---|
| Degree & Branch | B.Tech Artificial Intelligence & Data Science | Semester V |
| Subject Code & Name | CS3807 – Deep Learning Laboratory | AY: 2026–27 |

## 1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

## 2. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \qquad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = -\sum_i y_i \log(\hat{y}_i)$$

Baseline architecture used:

$$784 \rightarrow \text{Dense(128, ReLU)} \rightarrow \text{Dense(64, ReLU)} \rightarrow \text{Dense(10, Softmax)}$$

## 3. Dataset

Dataset: Fashion-MNIST
Training Images: 60,000
Testing Images: 10,000
Classes: 10
Image Size: $28 \times 28$

Fashion-MNIST images are stored as $28 \times 28$ matrices. Dense layers require a one-dimensional feature vector, so each image was flattened from $28 \times 28$ into a vector of length 784, and pixel values were normalized from the raw 0–255 range to $[0, 1]$ before training.

## 4. Source Code

The complete source code is available in the accompanying GitHub repository: `https://github.com/Ankith-Davey/Deep-Learning-Lab-Exp2-MLP`

## 5. Hyperparameter Optimization

Automated hyperparameter optimization was performed using RandomizedSearchCV together with the SciKeras wrapper, searching over the following space:

| Hyperparameter | Candidate Values |
|---|---|
| Hidden Layers | 1, 2, 3 |
| Hidden Neurons | 32, 64, 128, 256 |
| Learning Rate | 0.1, 0.01, 0.001 |
| Batch Size | 16, 32, 64, 128 |
| Epochs | 10, 20, 30 |
| Optimizer | SGD, Adam, RMSProp |
| Activation Function | ReLU, Tanh, Sigmoid |
| Dropout Rate (Optional) | 0.0, 0.2, 0.5 |

A baseline MLP was first built and trained on the full 60,000-image training set using the architecture in Section 2. RandomizedSearchCV then sampled 15 combinations from the search space above, evaluated using 5-fold cross-validation on a 15,000-image stratified subsample of the training data (for compute-time reasons). The best combination found was recorded, and a new model was retrained on the full 60,000-image training set using those hyperparameters, then evaluated on the test set and compared against the baseline.

## 6. Results

**Best Hyperparameters**

| | |
|---|---|
| Hidden Layers | 2 |
| Hidden Neurons | 256 |
| Learning Rate | 0.001 |
| Batch Size | 16 |
| Optimizer | RMSprop |
| Activation Function | Tanh |
| Epochs | 30 |
| Dropout | 0.2 |
| Cross-validation Accuracy | 0.8613 |
| Testing Accuracy | 0.8764 |

**Performance Comparison**

| Metric | Baseline | Optimized |
|---|---|---|
| Accuracy | 0.8809 | 0.8764 |
| Precision | 0.8828 | 0.8782 |
| Recall | 0.8809 | 0.8764 |
| F1-score | 0.8813 | 0.8761 |
| Training Time | 125.23s | 312.10s |

## 7. Plots with Inference

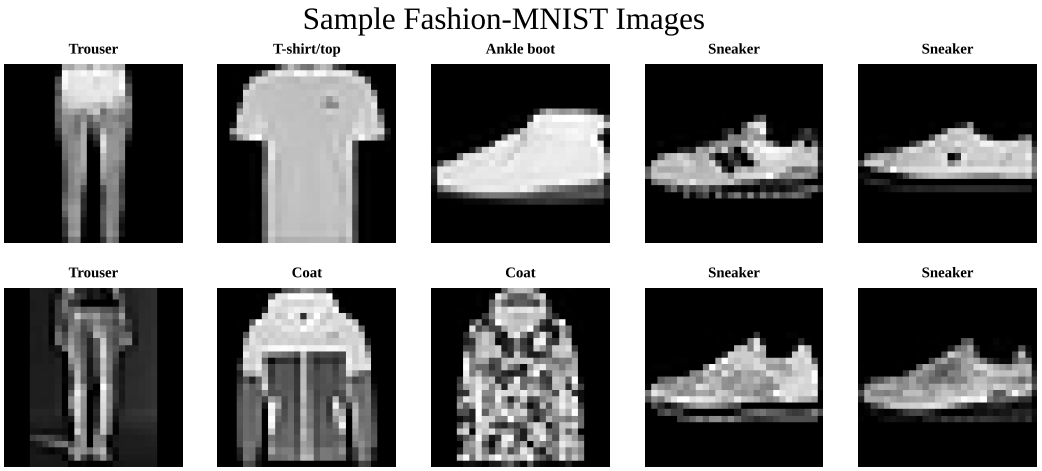


Figure 1: Sample images from the Fashion-MNIST dataset.

*The sample images consist of different types of fashion items such as trousers, t-shirts, bags, sneakers, etc. These are present in the Fashion-MNIST dataset, which is an image dataset with over 70000 (60000 for training and 10000 for testing) grayscale images.*

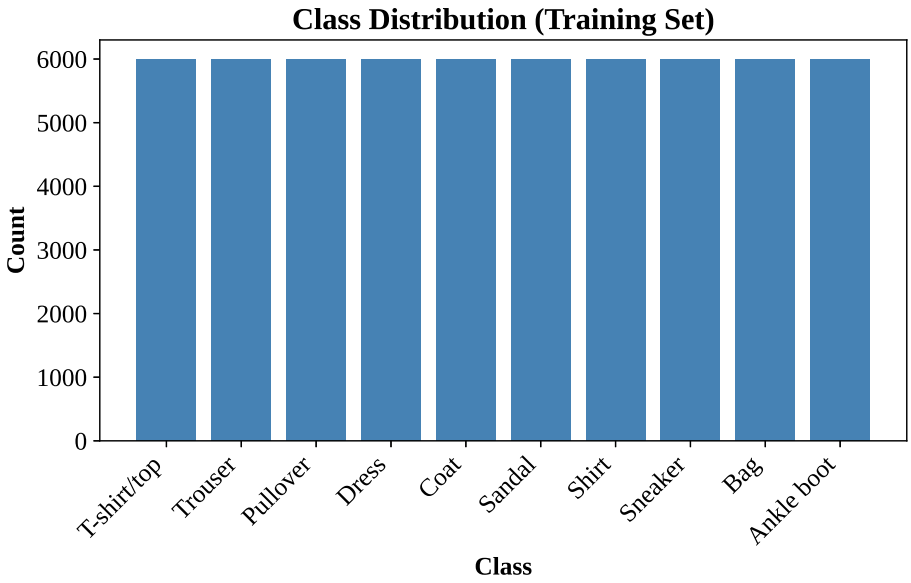


Figure 2: Class distribution of the training set.

*The training set consists of 60000 images in grayscale. There are a total of 10 classes, where each class has 6000 images. The classes include different types of trousers, sneakers, coats, etc.*

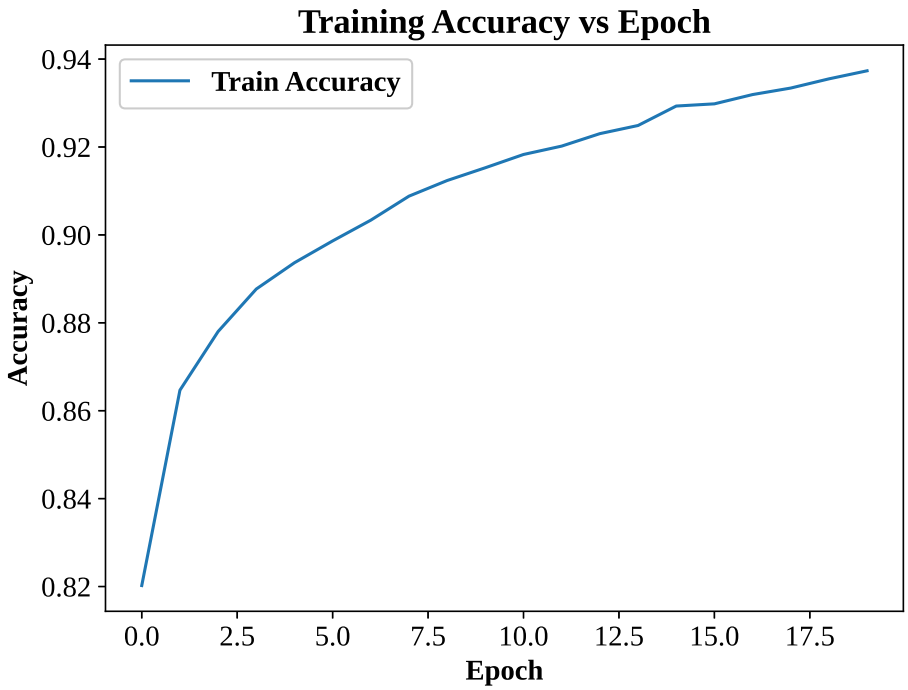


Figure 3: Training accuracy versus epoch.

*The training accuracy gradually increases as the number of epochs increases, meaning the model learns and updates weights in the right direction, with the highest training accuracy being 0.9373.*

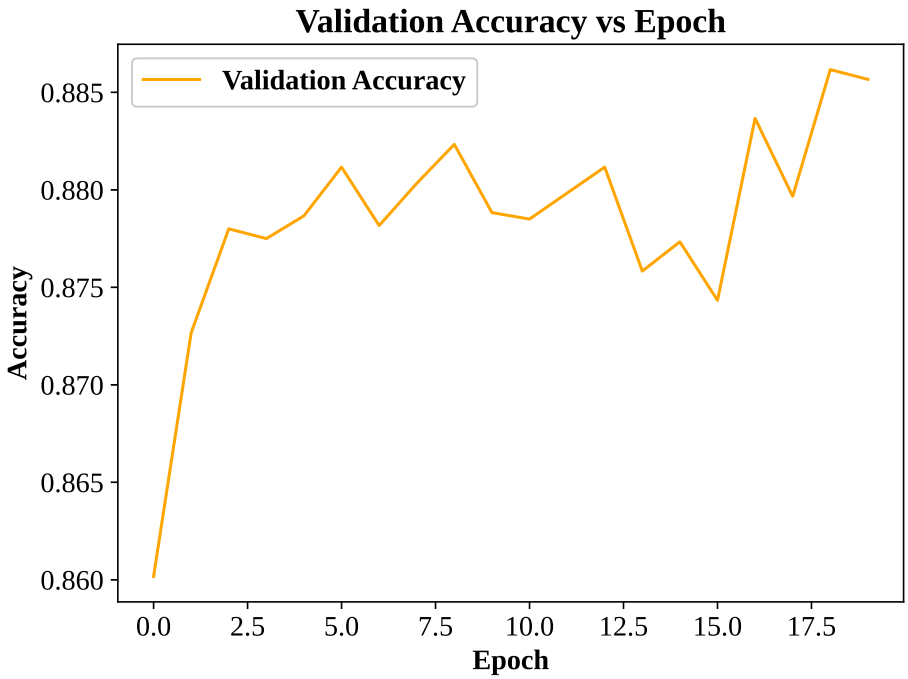


Figure 4: Validation accuracy versus epoch.

*The validation accuracy also increases gradually like the training accuracy, but the increase is less smooth than the training accuracy.*

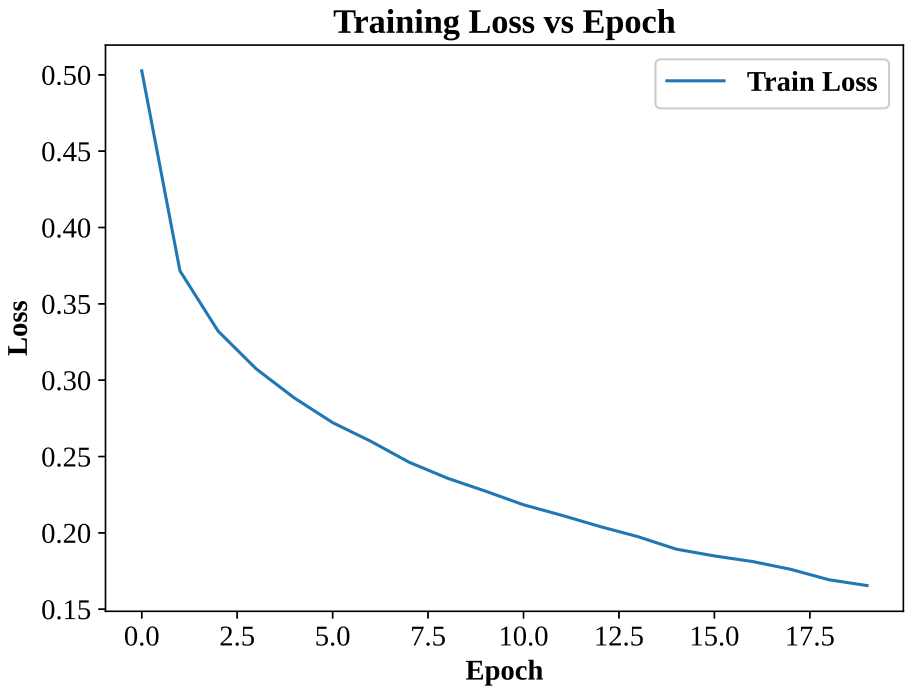


Figure 5: Training loss versus epoch.

*The training loss decreases as the number of epochs increases, which also supports the Training Accuracy vs Epoch graph. But the training loss never becomes zero, which means the model does not fully reach convergence by epoch 20.*

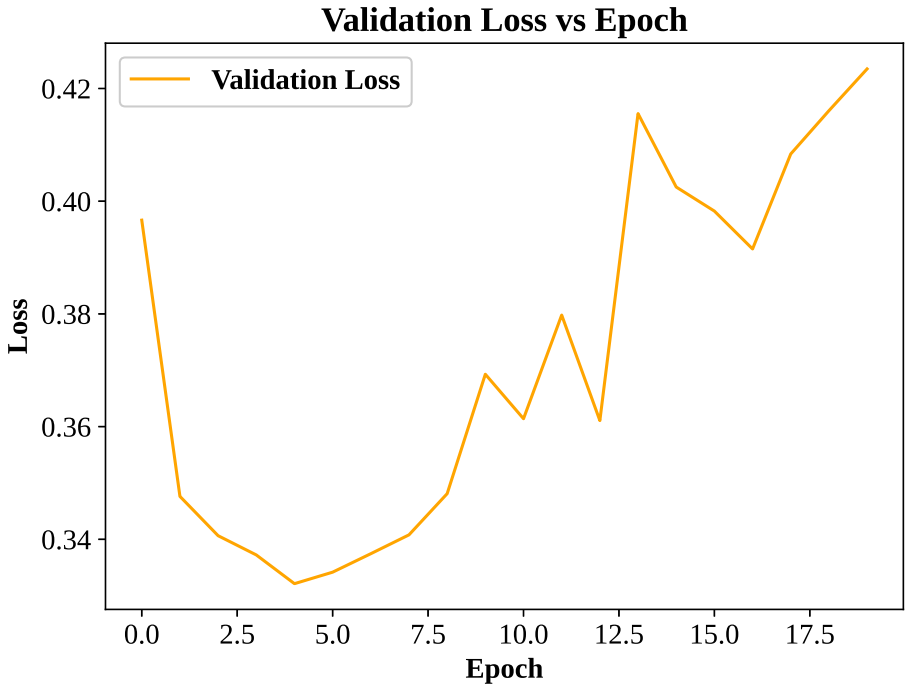


Figure 6: Validation loss versus epoch.

*The validation loss initially drops for the first 4 epochs, but keeps increasing with the number of epochs, with the loss rising to around 0.42 by the final epoch.*

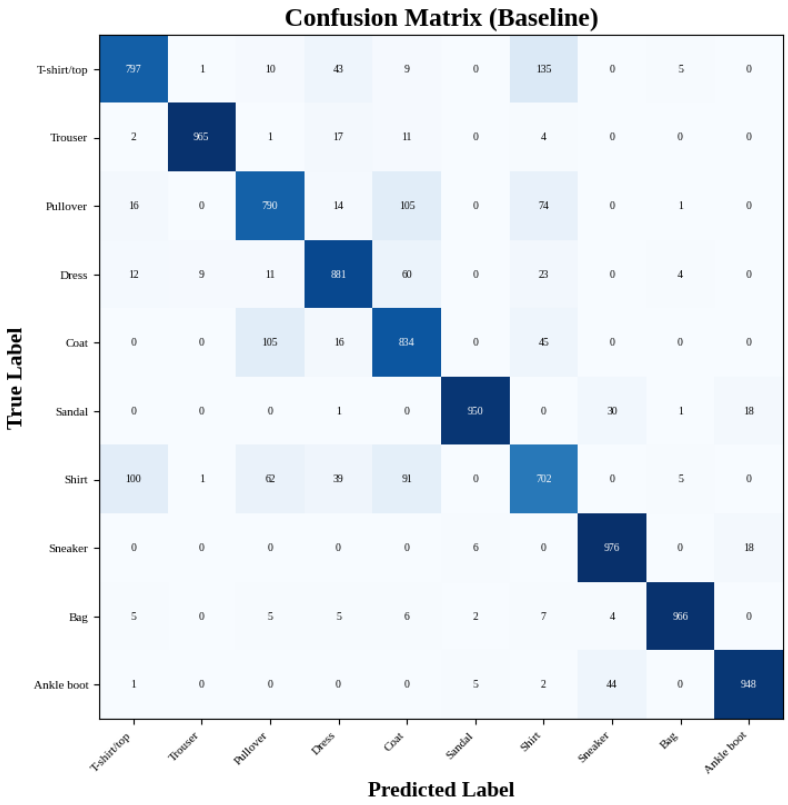


Figure 7: Confusion matrix of the baseline model on the test set.

*The confusion matrix shows that most classes in the Fashion-MNIST dataset were correctly classified, since the highest values were concentrated along the diagonal. The remaining errors occurred mainly between a few visually similar clothing categories, which suggests the model found these classes difficult to distinguish.*

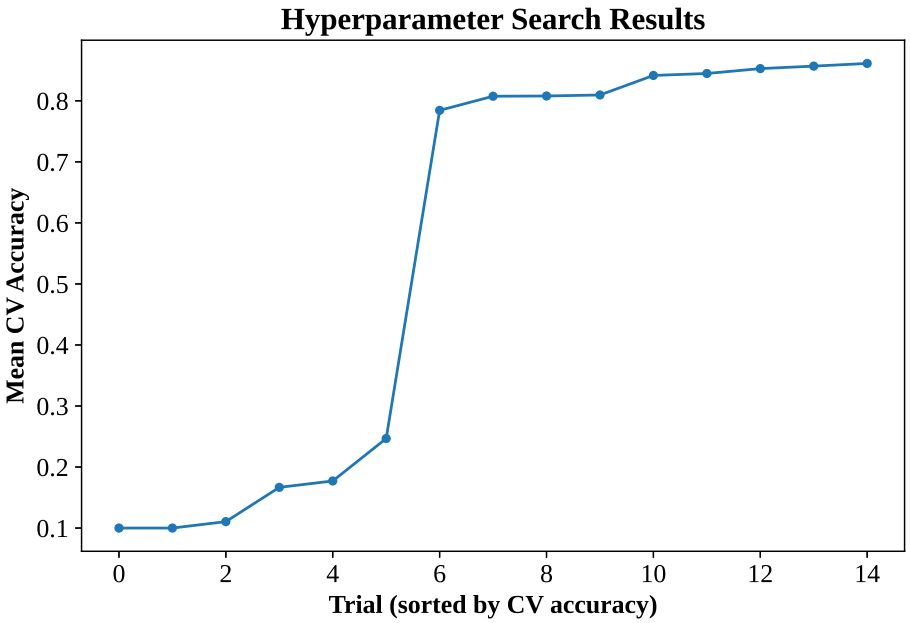


Figure 8: Hyperparameter search results across trials.

*The hyperparameter search graph showed a significant improvement in cross-validation accuracy once*

*an effective combination of parameters was identified. But after this initial increase, the remaining trials resulted only in small improvements, which indicates that the hyperparameter tuning mainly fine-tuned an already well-performing model rather than producing major performance gains.*

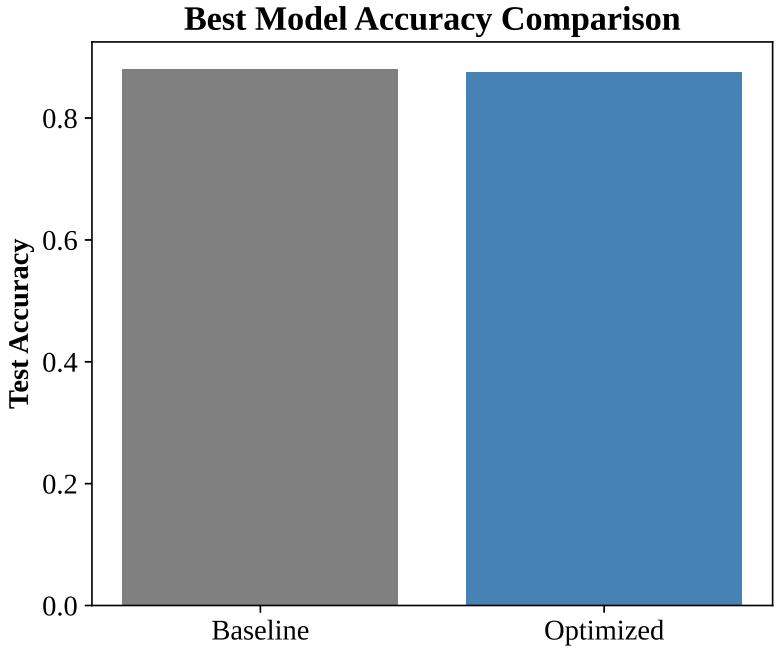


Figure 9: Best model accuracy comparison: baseline versus optimized.

*Although the optimized model achieved the highest cross-validation accuracy during hyperparameter tuning, the baseline model recorded a slightly higher test accuracy. This suggests that the optimized model did not generalize better to unseen (test) data, which highlights the fact that extensive hyperparameter tuning does not always guarantee improved performance.*

## 8. Discussion

1. **Which optimization method was used?**
   The RandomizedSearchCV method is used instead of GridSearchCV, which runs 15 randomly sampled combinations from the search space with 5-fold cross-validation, rather than testing every possible combination, which also reduces the compute time.
2. **Which hyperparameters were selected?**
   The best configuration emerged to be 2 hidden layers with 256 neurons per layer and optimizer as RMSprop; learning rate equal to 0.001, tanh activation, dropout rate 0.2, batch size 16, and 30 epochs, which gave a cross-validation accuracy of 0.8613.
3. **Did optimization improve performance?**
   No, the optimization did not improve performance on the actual test set. The optimized model scored an accuracy of 87.64%, which was slightly below the baseline model's accuracy of 88.09%. This was because the search itself ran on a 15,000-image subsample of the training data rather than the full 60,000 images, due to compute-time reasons, so the "best" hyperparameters were the best for that smaller subsample and not necessarily for the full dataset.
4. **Which hyperparameter had the greatest impact?**
   The hyperparameters which had the greatest impact on model performance were the activation function together with the learning rate. The large jump in cross-validation accuracy indicates that selecting an appropriate activation function and learning rate enables the convergence of the model, while changes

to the remaining hyperparameters mainly provided incremental improvements. This is also supported by the best-performing configuration, which used tanh as its activation function with a learning rate of 0.001, achieving a mean CV accuracy of 86.13%.

5. **Compare Grid Search and Randomized Search.**
   Grid Search tests every possible combination in the search space, which is exhaustive in nature but also guarantees the best combination within that grid. The problem is that it becomes extremely slow as the number of hyperparameters grows. For example, in this experiment, the search space has $3 \times 4 \times 3 \times 4 \times 3 \times 3 \times 3 \times 3 = 11{,}664$ combinations, which would be computationally difficult to fully grid-search with 5-fold CV. Randomized Search instead samples a fixed number of random combinations, trading a guarantee of finding the true best combination for a much faster and more practical search.
6. **Recommend the best MLP configuration.**
   The original baseline configuration (784→128→64→10, Adam optimizer, ReLU activation, batch size 32, 20 epochs) is a better practical choice, as it scored higher on the actual test set and trained in less than half the time. But with further optimization, running the hyperparameter search on the full training set (60,000 images) rather than 15,000 images may produce a configuration that generalizes better to the final model.

## 9. Conclusion

This experiment involved training an MLP model for image classification on the Fashion-MNIST dataset. The baseline model which was a simple 3-layer network achieved an accuracy of 88.09% test accuracy. An automated hyperparameter search was run to try and improve on that baseline but it gave a model with slightly less accuracy (87.64%). But this is because the search ran only on a subsample of 15,000 images instead of the full 60,000 due to the constraint of computation time. So the best combination it found was the best combination for that smaller subset of data and not necessarily for the full dataset. The accuracy and loss curves also showed the baseline model was slightly overfitting, since training accuracy kept climbing past 93% while validation performance leveled off and also got a little worse in the later epochs. So overall, this experiment proved that running an optimization process doesn't automatically mean a better model for this given subsample

## 10. References


1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.